

Banker's Algorithm

```
#include<stdio.h>
#include<stdbool.h>

int main()
{
    int allocation [5][4] = {
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4},
        {0,6,3,2},
        {0,0,1,4}
    };

    int max [5][4] = {
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6},
        {0,6,5,2},
        {0,6,5,6}
    };

    int need[5][4];

    int available[4] = {1,5,2,0};

    int safeAllocations = 0;

    bool isComplete[5] = {false};

    int safeSequence [5] = {};
```

```
for(int i = 0;i<5;i++)
{
    for(int j=0;j<4;j++)
    {
        need[i][j] = max[i][j]-allocation[i][j];
    }
}

for(int i=0;i<5;i++)
{
    for(int j=0;j<5;j++)
    {
        bool rowSafe = true;
        if(isComplete[j]==false)
        {

            for(int k=0;k<4;k++)
            {
                if(available[k]<need[j][k])
                {
                    rowSafe = false;
                }
            }

            if(rowSafe==true)
            {
                for(int k=0;k<4;k++)
                {
                    available[k]+=allocation[j][k];
                    isComplete[j]=true;
                }
            }
        }
    }
}
```

```
        }
    }
}

if(safeAllocations==5)
{
    printf("The system is in safe state.");
    printf("\nSafe Sequence => ");
    for(int i=0;i<safeAllocations;i++)
    {
        printf("%d, ", safeSequence[i]);
    }
}else{
    printf("The system is in unsafe state.");
}

return 0;
}
```

Output:

```
[Running] cd "g:\OS\" && gcc tempCodeRunnerFile.c -o
tempCodeRunnerFile && "g:\OS\"tempCodeRunnerFile
The system is in safe state.
Safe Sequence => 0, 2, 3, 4, 1,
[Done] exited with code=0 in 1.45 seconds
```